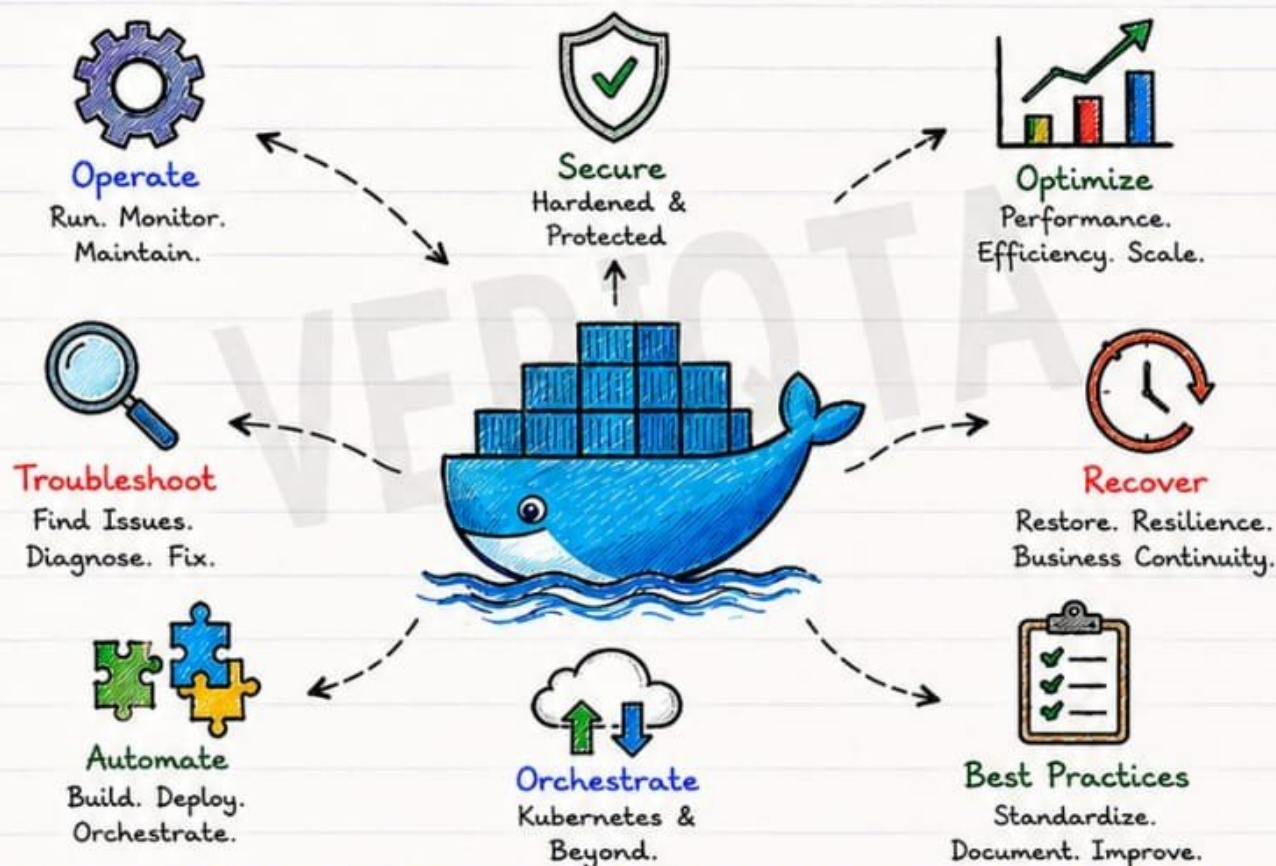


DOCKER HANDBOOK

Production Engineering & Troubleshooting



Real-world problems. Production-grade solutions.
Advanced troubleshooting. Engineering excellence.
This is where you build **reliable**, **scalable**, and **resilient** systems.



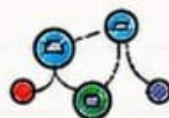
Monitoring
Observe & Alert



Logging
Centralize & Analyze



Automation
CI/CD & IaC



Orchestration
Kubernetes
Workloads



Security
Protect & Comply



Reliability
High Availability
& DR

Engineer it right. Run it well. Keep it reliable.



PRODUCTION DEPLOYMENT STRATEGIES

☆ Reliable deployments. Zero guesswork. Maximum uptime. ☆



Deploying to production is not just about running containers.
It's about delivering change safely, predictably, and repeatably.

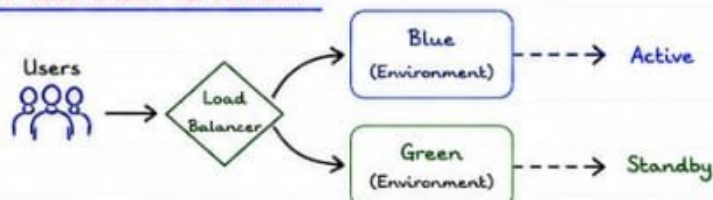
1. DEVELOPMENT vs PRODUCTION

DEVELOPMENT	PRODUCTION
- Fast & frequent changes - Local or shared environments - Minimal security - Debugging enabled - Lower resource needs - Failure is acceptable	- Stable & predictable releases - Isolated & highly available - Security hardened - Observability enabled - Right-sized resources - Failure is NOT acceptable



Goal: Move from fast & flexible (dev) to stable & reliable (prod).

2. BLUE-GREEN DEPLOYMENT

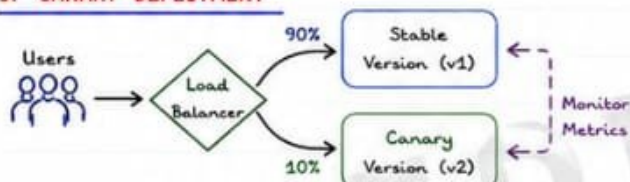


- Two identical environments: Blue (live) & Green (standby).
- Deploy to Green while Blue serves traffic.
- Switch traffic to Green when ready.
- Instant rollback by switching back to Blue.



Use Case: Large applications where downtime must be zero and rollback must be instant.

3. CANARY DEPLOYMENT



- Release to a small % of users first.
- Monitor metrics, logs, and errors.
- Gradually increase traffic to the new version.
- Rollback if issues are detected.



Use Case: Risky changes, new features, or performance-sensitive updates.

4. ROLLING UPDATES



- Update containers one at a time.
- Old version stays available while new version rolls out.
- No downtime, but rollback is slower than Blue-Green.
- Works well with load balancers and health checks.



Use Case: Continuous delivery with minimal risk and no full environment duplication.

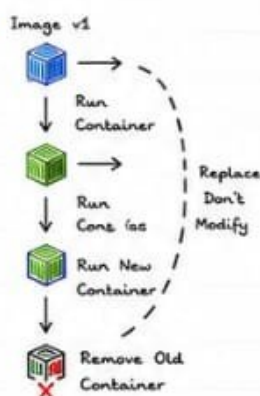
5. IMMUTABLE INFRASTRUCTURE

- Never change a running container.
- Build a new image.
- Deploy new containers.
- Destroy old containers.
- Consistent, repeatable, and reliable.



Benefits:

- ✓ No configuration drift
- ✓ Easy rollback
- ✓ Better security
- ✓ Predictable behavior



6. DEPLOYMENT PIPELINE



Automate everything. Reduce human error.
Every deployments should be repeatable.

7. PRODUCTION DEPLOYMENT WORKFLOW



8. BEST PRACTICES

- ✓ Always test in staging first.
- ✓ Use health checks on all services.
- ✓ Monitor metrics and logs during rollout.
- ✓ Have a rollback plan ready.
- ✓ Automate deployments with CI/CD.
- ✓ Keep configurations in version control.
- ✓ Follow the principle: Deploy Small. Verify. Scale.



The best deployments are boring.
Boring deployments mean your system is reliable.





DOCKER LOGGING DEEP DIVE

★ Logs are your system's memory. Collect them. Centralize them. Use them. ★



In production, good logging = fast troubleshooting, better insights, and happier users.

1. LOGGING DRIVERS

Docker uses logging drivers to capture container stdout and stderr and send them to different destinations.

DRIVER	DESCRIPTION	BEST FOR
json-file	Default driver. Logs written in JSON format on the host filesystem.	Dev/Test, small setups
journald	Send logs to systemd journal.	Systemd based Linux hosts
syslog	Send logs to a syslog server.	Traditional syslog environments
fluentd	Send logs to Fluentd for processing.	Flexible log pipelines
gelf	Send logs in GELF format to Graylog.	Graylog environments
awslogs	Send logs to Amazon CloudWatch Logs.	AWS environments
splunk	Send logs directly to Splunk HEC.	Splunk monitoring

2. DRIVER OVERVIEW

• json-file



Stored at `/var/lib/docker/containers/<id>/<id>-json.log`.

• journald



Integrated with systemd journal. Use: `journalctl -u docker`

• syslog



Send to remote syslog server. Use: 514/UDP or 514/TCP

• fluentd



Forward to Fluentd agent. Great for complex pipelines.

• gelf



Graylog Extended Log Format. Use with Graylog.

• awslogs



Stream logs to CloudWatch Logs. Region, log group, stream.

• splunk



Send logs to Splunk HTTP Event Collector (HEC).

3. CONFIGURING LOGGING DRIVER

--log-driver example:

```
docker run -d --name app \
  --log-driver=json-file \
  --log-opt max-size=10m \
  --log-opt max-file=5 \
  myapp:latest
```

Description:

- ✓ max-size: 10MB per file
- ✓ max-file: keep last 5 files

4. LOG ROTATION

Why rotate logs?

- ✓ Prevent disk full
- ✓ Improve performance
- ✓ Easier management
- ✓ Comply with retention policies



json-file rotation options:

```
--log-opt max-size=10m
--log-opt max-file=5
--log-opt compress=true
```

5. CENTRALIZED LOGGING

- ✓ Collect logs from all containers and hosts.
- ✓ Store in a central system.
- ✓ Search, analyze, alert and visualize.
- ✓ Essential for production visibility.

Popular Choices



ELK Stack
(Elasticsearch, Logstash, Kibana)



Grafana Loki



Graylog

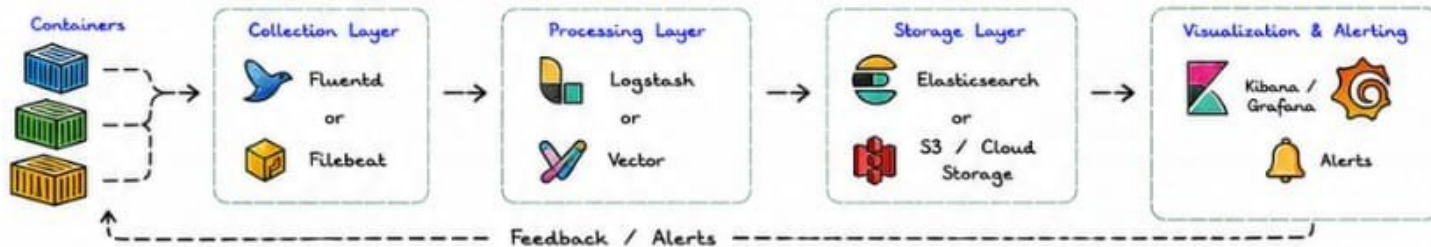


Azure Monitor



CloudWatch Logs

6. PRODUCTION LOGGING ARCHITECTURE



7. PRODUCTION BEST PRACTICES

- ✓ Use a logging driver suitable for your environment.
- ✓ Always configure log rotation.
- ✓ Centralize logs. Don't rely on local files.
- ✓ Include timestamps and container metadata.
- ✓ Monitor log shipping pipeline health.
- ✓ Secure log data in transit and at rest.

Useful Commands

```
docker logs <container>
docker logs -f <container>
docker inspect <container>
--format='{{.HostConfig.LogConfig}}'
journalctl -u docker
```

8. QUICK EXAMPLE (awslogs)

```
docker run -d --name app \
  --log-driver=awslogs \
  --log-opt awslogs-group=my-app-logs \
  --log-opt awslogs-region=us-east-1 \
  --log-opt awslogs-stream=app \
  myapp:latest
```



Good logging is not optional in production.
If you can't see it, you can't fix it.





DOCKER MONITORING

★ You can't improve what you don't measure. Monitor everything that matters. ★



Monitoring gives you visibility, early warnings,
and data to keep your containers healthy and your users happy.

1. CADVISOR

- Runs inside your environment.
- Collects container & host metrics.
- Exposes metrics for Prometheus.
- Lightweight and easy to deploy.
- Metrics: CPU, Memory, Network, Filesystem, Processes.



2. PROMETHEUS

- Time-series database.
- Scrapes metrics from targets.
- Powerful query language (PromQL).
- Alerting and rule engine.
- Pull-based monitoring system.



Prometheus

3. GRAFANA

- Visualization & dashboards.
- Connects to Prometheus.
- Rich charts and graphs.
- Alerts, annotations, templating.
- Beautiful + powerful UI.



Grafana

4. METRICS (What we collect)

CATEGORY	EXAMPLES OF METRICS
CPU	cpu_usage_seconds_total, cpu_system_seconds, cpu_user_seconds
MEMORY	memory_usage_bytes, memory_limit_bytes, memory_cache_bytes
NETWORK	network_receive_bytes_total, network_transmit_bytes_total, network_packets_total
DISK / FS	fs_usage_bytes, fs_limit_bytes, fs_reads_total, fs_writes_total
CONTAINER	container_start_time_seconds, container_last_seen, container_restart_count

5. DASHBOARDS

Common Dashboard Panels

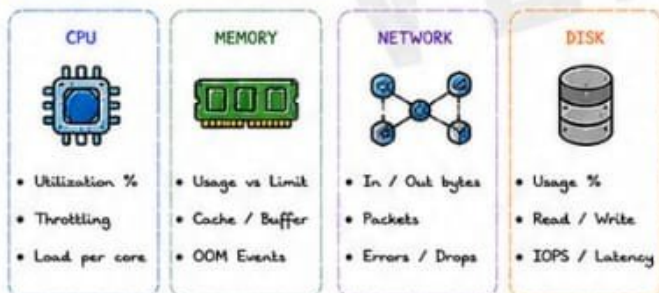
- ☒ CPU Usage (per container / host)
- ☒ Memory Usage
- ☒ Network In / Out
- ☒ Filesystem Usage
- ☒ Container Restart Count
- ☒ Top Containers by Resource Usage
- ☒ Alerts & Annotations

Dashboard Goals

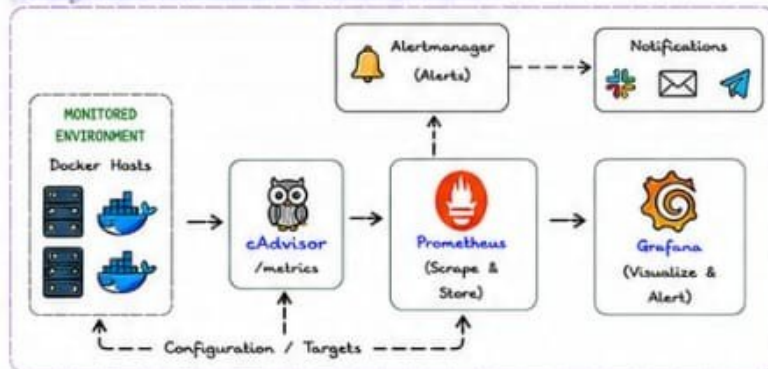
- ☒ Quick overview
- ☒ Drill down when needed
- ☒ Spot anomalies fast
- ☒ Act before users notice



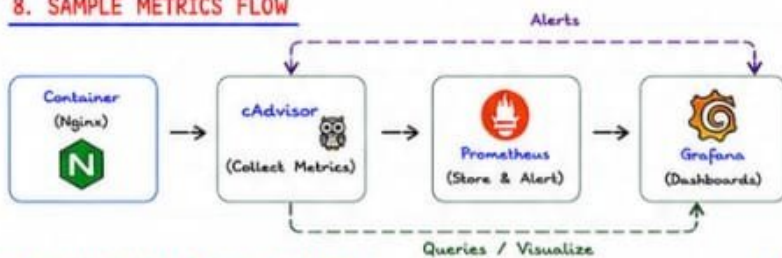
6. KEY RESOURCE AREAS



7. CONTAINER MONITORING ARCHITECTURE



8. SAMPLE METRICS FLOW



9. BEST PRACTICES

- ☒ Monitor everything important, not everything possible.
- ☒ Set meaningful alerts, not noisy alerts.
- ☒ Use labels to identify environments & services.
- ☒ Keep dashboards simple and actionable.
- ☒ Review and tune alerts regularly.
- ☒ Retain metrics long enough for troubleshooting.
- ☒ Test your monitoring & alerting pipeline.

10. QUICK EXAMPLE (CPU ALERT)



11. USEFUL PROMETHEUS QUERIES

- `rate(container_cpu_usage_seconds_total[5m])` # CPU usage
- `container_memory_usage_bytes` # Memory usage
- `rate(container_network_receive_bytes_total[5m])` # Network In
- `rate(container_network_transmit_bytes_total[5m])` # Network Out
- `container_fs_usage_bytes` # Filesystem usage



Good monitoring doesn't just detect problems,
it prevents them.
Observe. Understand. Act.





HEALTH CHECKS

★ Know when your containers are healthy. Recover fast when they are not. ★



Health checks help Docker detect unhealthy containers so you can react, restart, or replace automatically.

1. HEALTHCHECK INSTRUCTION

Define a command inside your Dockerfile to check if your container is healthy.

```
HEALTHCHECK --interval=30s \
  --timeout=5s \
  --retries=3 \
  --start-period=40s \
  CMD curl -f http://localhost:8080/health || exit 1
```



- `--interval` : how often to run the check
- `--timeout` : how long to wait for a response
- `--retries` : consecutive failures before unhealthy
- `--start-period` : grace period before counting failures

2. DOCKER INSPECT HEALTH

Use docker inspect to view health status and details.

```
$ docker inspect --format='{{json .State.Health}}' <container_id>
```

```
{
  "Status": "healthy",
  "FailingStreak": 0,
  "Log": [
    {
      "Start": "2024-06-10T10:15:30.123Z",
      "End": "2024-06-10T10:15:30.456Z",
      "ExitCode": 0,
      "Output": "HTTP 200 OK"
    }
  ]
}
```

STATUS VALUES

- starting (initializing)
- healthy (all good)
- unhealthy (needs attention)

3. LIVENESS

Is the container running?
Detects deadlocks, crashes, or stuck processes.



If liveness fails:
→ Container is restarted (if restart policy allows).

4. READINESS

Is the application ready to serve traffic?
Used by load balancers and orchestration.



If readiness fails:
→ Remove from load balancer
→ Try again later

5. STARTUP CHECKS

Give slow-starting apps time to warm up before checks are enforced.



Use `--start-period` to avoid false failures during startup.

6. RETRY LOGIC

Failures are not instant. Docker uses retry logic to avoid flapping.



Only after N consecutive failures container is marked unhealthy.

7. HEALTH CHECK FAILURES



Common reasons:

- Service down or crashed
- Port not listening
- Slow responses / timeouts
- Resource exhaustion
- Misconfigured command

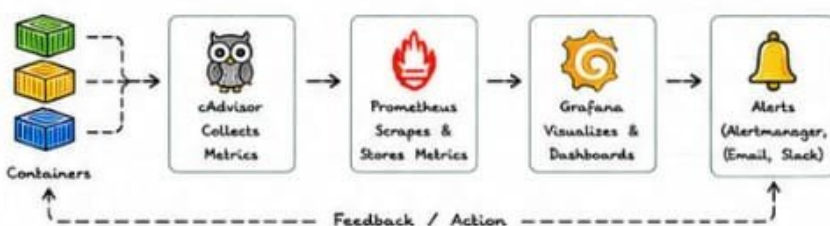


8. BEST PRACTICES

- ✓ Use meaningful health checks that verify real dependencies.
- ✓ Keep checks lightweight and fast.
- ✓ Use appropriate intervals and timeouts.
- ✓ Set a sensible start-period for slow apps.
- ✓ Monitor and alert on unhealthy containers.
- ✓ Test health checks before going to production.
- ✓ Don't rely only on container running = healthy.



9. CONTAINER MONITORING ARCHITECTURE



10. EXAMPLE HEALTH CHECK

```
# Dockerfile
HEALTHCHECK --interval=30s --timeout=5s \
  --retries=3 --start-period=40s \
  CMD curl -f http://localhost:8080/health || exit 1

# docker run
docker run -d --name myapp \
  -p 8080:8080 myapp:latest

# check health
docker inspect --format='{{json .State.Health}}' myapp
```



Healthy containers. Happy users. Stable systems.
Check early. Detect fast. Recover automatically.





RESTART POLICIES

★ Keep your containers alive. Recover fast. Stay available. ★



Restart policies control what happens when a container stops or crashes. Choose the right policy for reliability and stability.

1. RESTART POLICY OPTIONS

POLICY	DESCRIPTION	RESTARTS WHEN?	EXAMPLE COMMAND
no	Never restart automatically.	Never.	<code>--restart no</code>
on-failure[:max-retries]	Restart only if container exits with a non-zero code.	Container exits with failure code (1-255).	<code>--restart on-failure:5</code>
always	Always restart automatically.	Always, regardless of exit code.	<code>--restart always</code>
unless-stopped	Always restart unless the container was manually stopped.	Always, except when stopped by user.	<code>--restart unless-stopped</code>



Default:

If not specified, Docker uses "no".



2. FAILURE RECOVERY

How containers recover from failures.



Example:

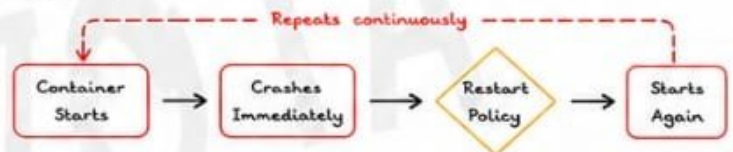
```
$ docker run --name app \
  --restart on-failure:3 myapp
```

- If app fails 3 times → container stops.
- If it succeeds → continues running.



3. RESTART LOOPS

When a container keeps crashing and restarting repeatedly.



Common Causes:

- Application bug
- Missing dependencies
- Bad configuration
- Insufficient resources

How to Detect:

- `docker ps` → STATUS (restarting)
- `docker events`
- `docker logs <container>`

4. PRODUCTION RECOMMENDATIONS

SCENARIO	RECOMMENDED POLICY
Web servers, APIs	always
Databases, stateful apps	unless-stopped
Batch jobs, one-time tasks	no
Worker services, queues	on-failure:5



Tip: Use health checks + restart policies together for maximum reliability.

5. POLICY BEHAVIOR SUMMARY

POLICY	Exit Code 0 (Success)	Exit Code 1-255 (Failure)	Manual Stop	Docker Daemon Restart
no	X (No restart)	X (No restart)	Stopped	Will not restart
on-failure[:max]	X (No restart)	✓ (Restart up to max times)	Stopped	Will not restart
always	✓ (Restart)	✓ (Restart)	Stopped	Will restart
unless-stopped	✓ (Restart)	✓ (Restart)	Stopped	Will restart (unless stopped)

6. QUICK EXAMPLES

No Restart

```
$ docker run --name test \
  --restart no myapp
```

Use for one-off tasks.



On-failure (3 retries)

```
$ docker run --name api \
  --restart on-failure:3 myapi
```

Restarts only on failure, up to 3 times.



Always Restart

```
$ docker run --name web \
  --restart always myweb
```

Always restarts. Best for web services.



Unless-Stopped

```
$ docker run --name db \
  --restart unless-stopped mydb
```

Restarts unless you stopped it manually.



The best systems recover automatically.
The best engineers design them to do so.





RESOURCE LIMITS

☆ Control resources. Prevent chaos. Protect your host. ☆



Resource limits stop one noisy container from crashing the whole system.
Reserve what you need. Limit what you use.

1. CPU LIMITS

Limit how much CPU a container can use.

Docker Options

- `--cpus="N"` Limit to N CPUs
- `--cpu-quota` Microseconds of CPU time allowed
- `--cpu-period` Period in microseconds (default 100000)
- `--cpu-shares` Relative weight vs other containers

Example

```
docker run -d \
  --name app \
  --cpus="1.5" \
  --cpu-shares=512 \
  myapp:latest
```



3. SWAP

Control swap usage for containers.

Docker Options

- `--memory-swap` Total memory (mem + swap)
- `--memory-swappiness` (Linux only)
0 = disable swap
100 = prefer swap

Example

```
docker run -d \
  --name app \
  -m 512m \
  --memory-swap 1g \
  --memory-swappiness=10 \
  myapp:latest
```



5. OOM KILLER

When memory is exhausted, the OOM Killer terminates the container.

How it works

- ✓ Container exceeds memory limit
- ✓ Kernel OOM Killer is triggered
- ✓ Container is killed to protect the host
- ✓ Docker restarts container if restart policy is set



7. PERFORMANCE BALANCING

Balance performance, stability, and cost.



Performance

- Give enough CPU/Memory
- Avoid throttling
- Monitor metrics



Stability

- Set limits
- Prevent noisy neighbors
- Avoid host crashes

Cost Efficiency

- Right-size containers
- Avoid over-provisioning
- Use reservations wisely

2. MEMORY LIMITS

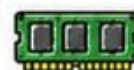
Limit how much memory a container can use.

Docker Options

- `-m, --memory` Hard memory limit
- `--memory-reservation` Soft (guaranteed) memory
- `--memory-swap` Total memory (memory + swap)

Example

```
docker run -d \
  --name app \
  -m 512m \
  --memory-reservation 256m \
  --memory-swap 1g \
  myapp:latest
```



4. PIDS

Limit the number of processes a container can create.

Docker Option

- `--pids-limit` Max number of PIDs

Example

```
docker run -d \
  --name app \
  --pids-limit=256 \
  myapp:latest
```



6. RESOURCE RESERVATIONS

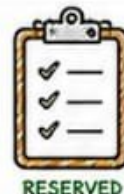
Reserve resources to guarantee performance.

Docker Options

- `--cpus (no limit)` Reserve CPU
- `--cpu-shares` Relative CPU weight
- `--memory-reservation` Reserve memory

Example

```
docker run -d \
  --name app \
  --cpus="1" \
  --cpu-shares=512 \
  --memory="1g" \
  --memory-reservation 512m \
  myapp:latest
```



8. RESOURCE LIMIT CHEAT SHEET

Area	Limit Option	What it Controls	Example
CPU	<code>--cpus="N"</code>	Total CPU cores	<code>--cpus="1.5"</code>
CPU	<code>--cpu-shares</code>	Relative CPU weight	<code>--cpu-shares=512</code>
Memory	<code>-m, --memory</code>	Hard memory limit	<code>-m 512m</code>
Memory	<code>--memory-reservation</code>	Soft memory guarantee	<code>--memory-reservation 256m</code>
Swap	<code>--memory-swap</code>	Total memory (mem + swap)	<code>--memory-swap 1g</code>
Swap	<code>--memory-swappiness</code>	Swap usage preference	<code>--memory-swappiness=10</code>
PIDs	<code>--pids-limit</code>	Max number of processes	<code>--pids-limit=256</code>



PRO TIP

Start with limits in staging.
Tune based on real usage.



Golden Rule

Limit what you can.
Reserve what you must.
Monitor everything.



REMEMBER

No limits = risk.
Smart limits = reliability.



Right limits. Happy containers. Healthy systems.



DOCKER BUILDKIT

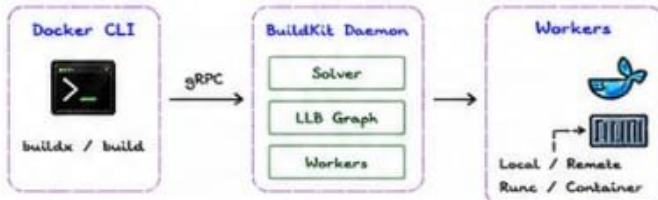
★ Smarter builds. Faster builds. Secure builds. ★



BuildKit is Docker's next-gen build engine. It's faster, more efficient, supports advanced features, and is now the default.

1. BUILDKIT ARCHITECTURE

BuildKit is a client-server architecture.



- CLI sends build request as LLB (Low-Level Build) to BuildKit.
- Solver builds an optimized graph of steps.
- Workers execute steps in parallel and return results.
- Output is cached and reused efficiently.

2. FASTER BUILDS

BuildKit significantly reduces build time.

- ✓ Smarter caching
- ✓ Parallel execution
- ✓ Minimal data transfer
- ✓ Incremental builds



Example Build Time Comparison

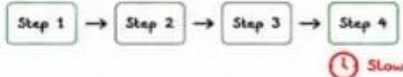
Builder	Cold Build	Rebuild (1layer changed)	Image Size
Legacy Builder	120s	90s	450MB
BuildKit	45s	8s	450MB

★ Up to 10x faster rebuilds!

3. PARALLEL BUILDS

BuildKit builds independent steps in parallel.

Legacy Builder (Sequential)



BuildKit (Parallel)



- Independent steps run at the same time. Less waiting, more building!

4. SECRET MOUNTS

Use secrets during build without storing them in the image.

```
# Dockerfile
RUN --mount=type=secret,id=mytoken \
  npm install
```

```
Build Command
docker buildx build \
  --secret id=mytoken,src=./token.txt \
  -t myapp:latest
```



- Secret is only available during the RUN step.
- Not stored in image layers.
- Safe and secure.

5. SSH MOUNTS

Use SSH keys to access private repos during build.

```
# Dockerfile
RUN --mount=type=ssh \
  git clone git@github.com:private/repo.git
```

```
Build Command
docker buildx build \
  --ssh default \
  -t myapp:latest
```



- For Git, private dependencies, and secure operations.
- Keys never end up in image.

6. CACHE MOUNTS

Persist build cache between builds.

```
# Dockerfile
RUN --mount=type=cache,target=/root/.npm \
  npm install
```

```
Build Command
docker buildx build \
  --progress=plain \
  -t myapp:latest
```



CACHE

- Cache is reused across builds.
- Speeds up package managers, compilers, downloads.

7. INLINE CACHE

Store cache metadata inside the image.

```
Build Command
docker buildx build \
  --build-arg BUILDKIT_INLINE_CACHE=1 \
  --cache-from myrepo/myapp:latest \
  --push -t myrepo/myapp:latest
```



- ★ Enables cache sharing across machines and CI/CD.

8. BUILD OPTIMIZATION

- ✓ Use .dockerignore
- ✓ Order instructions from least to most likely to change
- ✓ Combine RUN instructions
- ✓ Use cache mounts for package managers
- ✓ Use multi-stage builds
- ✓ Use specific base images (not :latest)
- ✓ Leverage inline cache in CI/CD

Example: Good Dockerfile

```
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm \
  npm ci --production
COPY . .
CMD ["node", "server.js"]
```



9. QUICK EXAMPLE WORKFLOW



Build smart. Build fast. Build safe.
BuildKit is your unfair advantage.





MULTI-STAGE BUILD OPTIMIZATION

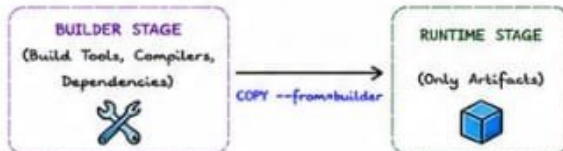
★ Build lean. Run clean. Ship secure. ★



Multi-stage builds let you use multiple FROM statements in your Dockerfile.
You keep the build tools, but ship only what you need to run.

1. BUILDER PATTERN

Use one stage to build/compile, another to run.
Keep build dependencies out of the final image.



- Benefits:
- ✓ Smaller images
 - ✓ Faster push/pull
 - ✓ Reduced attack surface
 - ✓ Cleaner runtime environment

3. DISTROLESS IMAGES

Images without package managers, shells, or extra tools.
Only your app and its runtime.

Why Distroless?

- ✓ Minimal attack surface
- ✓ No shell = harder to exploit
- ✓ Fewer CVEs
- ✓ Smaller size
- ✓ Designed for production

Common Distroless Variants

- `gcr.io/distroless/base`
- `gcr.io/distroless/cc`
- `gcr.io/distroless/python3`
- `gcr.io/distroless/nodejs`
- `gcr.io/distroless/java`



Note:

Distroless has no shell. Debugging is harder.
Use logs and proper monitoring.

2. RUNTIME IMAGES

Use minimal base images for the final stage.

Base Image Type	Examples	Size (approx)	Use Case
Full OS	<code>ubuntu:24.04</code>	100-200MB+	General purpose
Slim	<code>debian:bookworm-slim</code>	30-80MB	Good balance
Alpine	<code>alpine:3.19</code>	5-10MB	Lightweight (musl)
Distroless	<code>gcr.io/distroless/*</code>	<5MB	Production (minimal)



Always choose the smallest image that meets your needs.

4. IMAGE SIZE REDUCTION

Tips to reduce your final image size.

- ✓ Use multi-stage builds
- ✓ Use slim or distroless images
- ✓ Remove build dependencies
- ✓ Use `.dockerignore`
- ✓ Clean package cache in builder
- ✓ Copy only what you need
- ✓ Avoid unnecessary packages
- ✓ Use specific tags (not latest)
- ✓ Compress assets in builder

Example Size Comparison



5. SECURITY IMPROVEMENTS

Multi-stage + minimal images = more secure.

- ✓ Fewer vulnerabilities
- ✓ No build tools in runtime
- ✓ No package managers
- ✓ Run as non-root user
- ✓ Read-only filesystem ready
- ✓ Smaller attack surface



Security Best Practices

- Use distroless or slim images
- Drop unnecessary packages
- Run as non-root user
- Set read-only root filesystem
- Scan final image for CVEs
- Keep images updated

6. PRODUCTION EXAMPLES

Example: Node.js App

Dockerfile (Node.js)

```
# Builder Stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Runtime Stage
FROM gcr.io/distroless/nodejs20-debian11
WORKDIR /app
COPY --from=builder /app/dist ./dist
CMD ["server.js"]
```

Example: Go App

Dockerfile (Go)

```
# Builder Stage
FROM golang:1.22-alpine AS builder
WORKDIR /src
COPY . .
RUN go mod download
RUN CGO_ENABLED=0 go build -o app

# Runtime Stage (Distroless)
FROM gcr.io/distroless/static-debian11
COPY --from=builder /src/app /app
USER nonroot:nonroot
CMD ["/app"]
```



Result: Small, secure, and production-ready images.

COMPARISON: SINGLE STAGE vs MULTI-STAGE

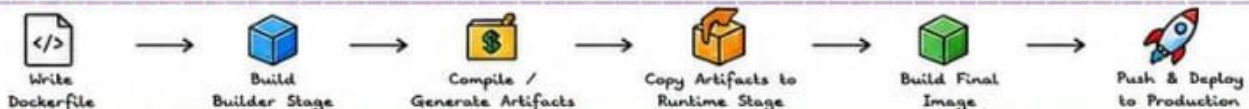
Aspect	Single Stage Build	Multi-stage Build	Benefit
Image Size	Large	Small	Faster push/pull, less storage
Attack Surface	Large	Small	More secure
Build Tools in Final?	Yes	No	Cleaner runtime
Performance	Slower	Faster	Less data transfer
Best For	Simple apps	Production apps	Scalable & secure



KEY TAKEAWAY

Build with everything.
Run with nothing extra.
That's the power of
multi-stage builds.

TYPICAL MULTI-STAGE WORKFLOW



Small images. Secure systems. Happy users.
Optimize your builds. Optimize your production.





IMAGE SCANNING WITH TRIVY

★ Scan early. Find issues. Fix fast. Ship safe. ★



Trivy is a fast, comprehensive, and easy-to-use vulnerability scanner for containers, filesystems, and repositories.

1. VULNERABILITY SCANNING

Scan container images, filesystems, and code repositories for known vulnerabilities and misconfigurations.

Trivy Can Scan:

- ✓ Container images
- ✓ Filesystem / directories
- ✓ Git repositories
- ✓ Kubernetes manifests
- ✓ SBOM (Software Bill of Materials)
- ✓ Cloud resources (AWS, GCP, Azure)



TRIVY

3. SEVERITY LEVELS

Trivy maps CVSS scores to severity levels.

SEVERITY	CVSS SCORE	COLOR	DESCRIPTION
CRITICAL	9.0 - 10.0	Red circle	Immediate action required
HIGH	7.0 - 8.9	Orange circle	High risk, fix soon
MEDIUM	4.0 - 6.9	Yellow circle	Moderate risk
LOW	0.1 - 3.9	Blue circle	Low risk
UNKNOWN	N/A	Grey circle	Not enough data

★ Focus on **HIGH** and **CRITICAL** first!

5. CI/CD INTEGRATION

Run Trivy in your pipeline to stop vulnerable images from reaching production.



Example: GitHub Actions

```
- name: Run Trivy vulnerability scanner
  uses: aquasecurity/trivy-action@v0.19.0
  with:
    image-ref: 'myapp:latest'
    format: 'table'
    severity: 'CRITICAL,HIGH'
    exit-code: '1'
    ignore-unfixed: true
```



✓ Fail the pipeline if **CRITICAL** or **HIGH** vulnerabilities exist.

7. PRODUCTION POLICIES

Define clear policies to keep production secure.

POLICY AREA	POLICY EXAMPLE	ACTION
Scan Coverage	All images must be scanned before push	Block push if scan not completed
Severity Threshold	Block CRITICAL and HIGH vulnerabilities	Fail build / block deployment
Base Image	Use only approved and maintained base images	Reject non-approved images
Scan Frequency	Rescan images daily in registry	Alert on new vulnerabilities
Exception Process	Document and approve exceptions	Review periodically
Reporting	Send reports to Slack / Email	Notify security team

Example Policy Config (trivy.yaml)

```
severity: ["CRITICAL", "HIGH"]
exit-code: 1
ignore-unfixed: true
vuln-type: ["os", "library"]
format: table
cache-dir: ".trivycache"
scanners: ["vuln", "config"]
```



2. CVEs

Trivy detects vulnerabilities using official vulnerability databases.

Example CVE Found by Trivy

CVE-2024-3094

Package: **openssl**

Installed Version: 1.1.1k-r0

Fixed Version: 1.1.1k-r1

Title: OpenSSL TLS vulnerability (x509 parsing)
Description: Openssl x509 certificate parsing vulnerability can lead to denial of service.

Published: 2024-05-21

CVSS Score: **7.5 (HIGH)**



VULNERABILITY
DATABASES

4. SCAN WORKFLOW

Typical workflow for scanning a Docker image.



Example Command

```
trivy image --format table myapp:latest
```

```
trivy image --severity HIGH,CRITICAL --exit-code 1 myapp:latest
```

★ --exit-code 1 fails the build if vulnerabilities are found.

6. REMEDIATION

Steps to fix vulnerabilities.

- ① Update base image to the latest secure version.
- ② Update vulnerable packages in Dockerfile.
- ③ Rebuild and rescan the image.
- ④ Verify fix in vulnerability report.
- ⑤ If risk is acceptable, add to .trivyignore with justification.

.trivyignore Example

```
# Ignore specific CVE
CVE-2023-12345

# Ignore by package
package:openssl

# Ignore by severity
severity:LOW
```



Always fix the root cause, not just ignore it.



Scan often. Fix always. **Secure** images. Safe production.
Trivy in CI, safe in prod.





DOCKER BENCH SECURITY

★ Secure your Docker hosts. Follow the benchmarks. Stay compliant. ★



Docker Bench for Security checks your Docker installation against the CIS (Center for Internet Security) Benchmark best practices.

1. CIS BENCHMARK

CIS provides hardened configuration guidelines for Docker to reduce risk and attack surface.



Center for
Internet Security

CIS Benchmark Goals

- ☒ Minimize attack surface
- ☒ Secure default settings
- ☒ Limit privilege escalation
- ☒ Protect container runtime
- ☒ Improve auditability & logging

3. HOST SECURITY

Secure the Docker host OS and environment.

CHECK AREA	BEST PRACTICE	WHY IT MATTERS
OS Updates	Keep OS updated	Patch known vulnerabilities
Firewall	Enable firewall	Restrict unauthorized access
User Accounts	Use non-root users	Prevent privilege abuse
SSH Access	Disable root login	Reduce brute force risk
File Permissions	Restrict sensitive files	Prevent info leakage
Audit & Logging	Enable system logging	Improve traceability



5. RUNTIME SECURITY

Secure how containers run in production.

RUNTIME BEST PRACTICES

- ☒ Drop unnecessary capabilities
- ☒ Run as non-root user
- ☒ Use read-only root filesystem
- ☒ Limit privileged containers
- ☒ Enable user namespace remapping
- ☒ Set resource limits (CPU, Memory, PIDs)
- ☒ Scan images for vulnerabilities



7. SECURITY AUDITING

Audit regularly and track improvements.



Sample Audit Command

```
$ docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock \
  docker/docke-bench-security
```

Audit Tips

- ☒ Automate in CI/CD pipeline
- ☒ Schedule monthly audits
- ☒ Compare scores over time
- ☒ Store reports for compliance



Secure by default. Audit by habit. Comply with confidence.
Security is a feature, not an afterthought.

2. DOCKER BENCH

Docker Bench for Security is a script that automates checks against the CIS Benchmark.

Run Docker Bench

```
$ docker run --rm -it \
  docker/docke-bench-security
```

Output Summary

```
Tests Performed : 100+
Passed           : ✓
Failed           : ✗
Warnings         : ⚠
Info             : ⓘ
```

Score Example



4. DAEMON SECURITY

Harden the Docker daemon configuration.

CHECK	RECOMMENDATION
Secure Socket	Use TLS for daemon socket
TCP Binding	Do not bind daemon to 0.0.0.0
Access Control	Limit who can access the daemon
Logging	Configure daemon logging
Storage Driver	Use secure storage driver (e.g., overlay2)
Default Ulimits	Set secure default ulimits



★ Never expose the Docker daemon to the public internet.
It gives root-equivalent access to the host.

6. COMPLIANCE

Docker Bench aligns with common compliance frameworks.



Example:

CIS Docker Benchmark → Helps meet ISO 27001 A.12.6.2
(Technical vulnerability management)



Example Report (Summary)

```
Total Tests : 112
Passed      : 86
Failed      : 18
Warnings    : 6
Score       : 76/100
Grade       : B
```





DOCKER SECRETS

☆ Protect secrets. Control access. Keep them out of images and env vars. ☆



Docker Secrets in Swarm mode provides a secure way to store and use sensitive data like passwords, tokens, and keys.

1. SWARM SECRETS

Secrets are encrypted at rest in the Raft log and distributed only to authorized services.

Key Points

- ✓ Managed by Docker Swarm
- ✓ Encrypted in transit (TLS) and at rest
- ✓ Mounted in tmpfs (in-memory filesystem)
- ✓ Not accessible to unauthorized containers
- ✓ Not persisted on container restart



SECURE
BY DESIGN

Create a Secret

```
$ echo "superSecret123" | \
  docker secret create db_password -
$ docker secret ls      # List secrets
```

3. SECRET ROTATION

Rotate secrets without downtime.



Example Rotation

```
$ echo "newPass12024" | docker secret create db_password_v2 -
$ docker service update --secret-rm db_password \
  --secret-add source=db_password_v2,target=db_password myapp
$ docker secret rm db_password      # After update completes
```

★ Always test after rotation!

5. BEST PRACTICES

- ✓ Never store secrets in images, env vars, or code
- ✓ Use least-privilege access and granular secrets
- ✓ Rotate secrets regularly
- ✓ Use external secret management system for large-scale (eg: HashiCorp Vault, AWS Secrets Manager)
- ✓ Audit who can create, read, and use secrets
- ✓ Limit number of secrets per service
- ✓ Use meaningful names and versioning (eg: db_pass_v2)



What NOT to do

- ✗ Don't commit secrets to Git
- ✗ Don't bake secrets in images
- ✗ Don't log secrets
- ✗ Don't over-share secrets

2. SECRET LIFECYCLE

Secrets follow a secure lifecycle from creation to removal.



Lifecycle Details

Stage	Description
Create	Admin creates secret using CLI / API
Store	Stored encrypted in Swarm Raft log
Distribute	Distributed to nodes running the service
Mount	Mounted in memory (tmpfs) inside container
Remove	Secret is removed when no longer needed

4. RUNTIME INJECTION

Secrets are mounted as files in `/run/secrets/<secret_name>`

Service Definition (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    secrets:
      - db_password
      - api_key

secrets:
  db_password:
    external: true
  api_key:
    external: true
```

Inside Container

```
$ ls -l /run/secrets
-rw-r--r-- 1 root root 32 May 20 10:01 db_password
-rw-r--r-- 1 root root 48 May 20 10:01 api_key

$ cat /run/secrets/db_password
superSecret123
```

Notes

- ✓ Mounted as files (read-only)
- ✓ Accessible only to the service
- ✓ Not visible in "docker inspect"
- ✓ Not in env vars, not in image



Secrets never appear in environment variables or logs unless your app prints them (don't do that!).

6. PRODUCTION EXAMPLES

Example 1: Database Password

```
$ echo "P@ssw0rd!" | docker secret create db_pass -
# Use in service
docker service create \
  --name webapp \
  --secret source=db_pass,target=db_password \
  myapp:latest
```

Inside App (read from file):

```
const fs = require('fs');
const dbPass = fs.readFileSync('/run/secrets/db_password',
  'utf8').trim();
```

Example 2: API Key

```
$ echo "sk_live_abc123" | \
  docker secret create api_key -
docker service create \
  --name worker \
  --secret source=api_key,target=apikey \
  worker:latest
```

Benefits in Production

- ✓ Secure credential management
- ✓ Easier rotation without downtime
- ✓ Reduced risk of secret leakage
- ✓ Compliance ready

SECRET FLOW IN SWARM



Remember:

Secrets are for runtime only. Keep them short-lived, rotated, and protected.



Right secrets. Right people. Right time.
Secure today. Strong tomorrow.





ROOTLESS DOCKER

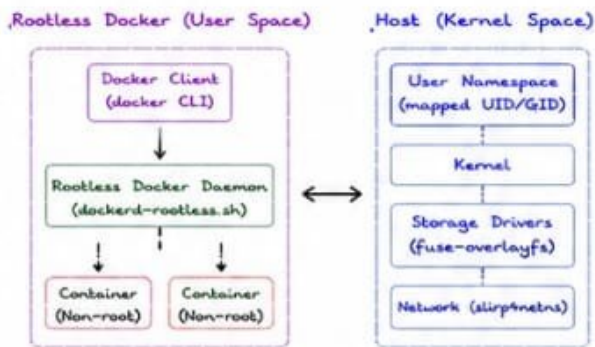
★ Run Docker without root. Less privilege. More security. ★



Rootless Docker allows you to run the Docker daemon and containers as a non-root user. It reduces the attack surface on the host.

1. ROOTLESS ARCHITECTURE

Rootless Docker runs the daemon in the user namespace of an unprivileged user.



- ✓ No root required for daemon or containers.
- ✓ Uses user namespaces, cgroups v2 (if available), slirpnetns, and fuse-overlays.



2. SECURITY BENEFITS

Significantly improves host security.

- ✓ No root daemon = less impact if compromised
- ✓ Containers run as non-root by default
- ✓ Daemon cannot modify critical system files
- ✓ Limits lateral movement on the host
- ✓ Safer multi-tenant / developer environments
- ✓ Reduces risk of privilege escalation



Impact Example

In **root mode**, a container escape could give attacker root access to the host.

In **rootless mode**, attacker gets access only to the user session, not the entire **host**.

Best for:

- Developer laptops
- CI runners
- Shared servers
- Non-privileged services
- Organizations with strict security policies



3. LIMITATIONS

Rootless Docker has some trade-offs.

- ✗ Cannot bind to privileged ports (< 1024)
- ✗ Lower network performance (slirpnetns)
- ✗ Some storage drivers not available (uses fuse-overlays)
- ✗ No access to certain kernel features
- ✗ Swarm mode not supported
- ✗ Less suitable for high performance workloads



4. INSTALLATION

Install Rootless Docker on Linux (example: Ubuntu).

Install Rootless Docker

1. Install dependencies

```
sudo apt update
sudo apt install -y uidmap slirpnetns fuse-overlays
```
2. Install Rootless Docker

```
curl -fsSL https://get.docker.com/rootless | sh
```
3. Start user session and enable service

```
dockerd-rootless-setup tool.sh install
```
4. Start Rootless Docker

```
systemctl --user start docker
```
5. Verify docker info

Notes

- Works best with cgroups v2.
- Requires Linux kernel 5.11+ (recommended).
- Login again or run "newgrp" for changes.



5. PRODUCTION CONSIDERATIONS

When to use Rootless Docker in production.

- ✓ Use on single-tenant or low-privileged servers
- ✓ Ensure cgroups v2 and kernel are compatible
- ✓ Monitor performance (network/storage overhead)
- ✓ Combine with least-privilege containers (non-root user)
- ✓ Use for build agents, sidecars, and auxiliary services
- ✓ Not recommended for Swarm or high throughput apps



Key Takeaway

★ Rootless Docker is not for every workload, but it's excellent for isolating risk.

6. COMPARISON WITH ROOT MODE

Rootless vs Root mode at a glance.

Feature	Rootless Docker	Root Mode (Default)
Daemon User	Non-root user	Root (UID 0)
Attack Surface	Lower	Higher
Container User	Non-root by default	Root unless changed
Privileged Ports	✗ (cannot bind <1024)	✓ (can bind)
Networking	slirpnetns (user space)	Native (bridge/host)
Storage Driver	fuse-overlays	overlay2, btrfs, etc.
Performance	Lower (some overhead)	Higher
Swarm Mode	✗ Not supported	✓ Supported
Best Use Case	Dev, CI, non-priv apps	Prod, high-perf, Swarm

Rootless Flow



Run containers. Protect hosts. Choose **Rootless**.
Secure by default. Safe by design.





DISTROLESS IMAGES

☆ Less is more. Build smaller, secure, and production-ready containers. ☆



Distroless images contain only your app and its runtime dependencies.
No package manager. No shell. No unnecessary bloat.

1. WHAT ARE DISTROLESS IMAGES?

Distroless images are minimal base images that contain only the application runtime and required system libraries. They do not include package managers, shells, or other userland utilities.

Key Characteristics

- ✓ No package manager (apk, apt, yum)
- ✓ No shell (bash, sh)
- ✓ No unnecessary utilities (curl, wget, ps, netstat)
- ✓ Built for a single purpose: run your app
- ✓ Smaller, faster, and more secure



3. IMAGE COMPARISON

ASPECT	ALPINE (Example)	DISTROLESS (Example)
Base Image	alpine:3.19	gcr.io/distroless/java17-debian11
Image Size	~ 120 MB	~ 19 MB
Shell	/bin/sh (available)	✗ None
Package Manager	apk (available)	✗ None
Utilities	curl, wget, ps, netstat, top...	✗ None
Attack Surface	Large	Minimal
CVE Exposure	Higher	Lower
Best For	Debugging & Dev	Production

2. SECURITY ADVANTAGES

Distroless images significantly improve your security posture.

Security Benefits

- ✓ Smaller attack surface
- ✓ Fewer binaries = fewer CVEs
- ✓ No shell = harder for attackers to move inside
- ✓ No package manager = cannot install malware
- ✓ Reduced risk of privilege escalation
- ✓ Easier to meet compliance requirements



SECURE
BY DESIGN

4. MINIMAL ATTACK SURFACE

Every package, tool, and shell increases your risk.

Attack Surface Analogy

Alpine image = a full toolbox

Distroless image = only the tool you need

Alpine Image



Distroless Image



★ Fewer components = fewer vulnerabilities = better security.

5. PRODUCTION USE CASES

Distroless images are ideal for production workloads.

Common Use Cases

- ✓ Microservices (Go, Java, Python, Node.js)
- ✓ gRPC / REST APIs
- ✓ Serverless container platforms
- ✓ Batch jobs & schedulers
- ✓ High-security enterprise applications
- ✓ Fintech, Healthcare, Government workloads



6. BEST PRACTICES

Follow these practices to safely adopt distroless images.

Best Practices

- ✓ Use multi-stage builds to copy only the artifacts
- ✓ Ensure your app logs to STDOUT/STDERR
- ✓ Do not rely on shell scripts inside the image
- ✓ Use non-root user (distroless images default to non-root)
- ✓ Scan images for vulnerabilities (Trivy, Grype, etc.)
- ✓ Test thoroughly in staging before production
- ✓ Keep images updated and rebuild regularly



PRODUCTION EXAMPLE: GO MICROSERVICE



Write
Go App



Build
Binary



Multi-stage
Docker Build



Distroless
Runtime Image



Push to
Registry



Run in
Production

Dockerfile (Multi-stage with Distroless)

```
# Stage 1: Build
FROM golang:1.22-alpine AS builder
WORKDIR /src
COPY . .
RUN go build -o app /cmd/server

# Stage 2: Runtime
FROM gcr.io/distroless/base-debian11
WORKDIR /app
COPY --from=builder /src/app /app/app
USER nonroot:nonroot
ENTRYPOINT ["/app/app"]
```

Why It's Better

- ✓ Image size reduced from ~120MB to ~19MB
- ✓ No shell for attackers to exploit
- ✓ No package manager to abuse
- ✓ Faster cold starts
- ✓ Lower CVE count
- ✓ Easier compliance (PCI-DSS, ISO 27001)

Example Image Sizes

Image	Size (approx)
alpine:3.19	~ 120 MB
debian:bookworm-slim	~ 80 MB
amazonlinux:2	~ 70 MB
distroless/base-debian11	~ 7 MB
distroless/java17-debian11	~ 19 MB



Distroless images = smaller, faster, safer.

Build lean. Run clean. Stay secure.





PRODUCTION DEBUGGING

★ Problems will happen. Fast debugging reduces downtime and saves money. ★



Debug like a pro. Observe → Understand → Isolate → Fix → Verify.
Stay calm. Follow the process.

1. LOGS

Logs show what happened in your container.

Common Commands

```
$ docker logs <container_id> # all logs
$ docker logs -f <container_id> # follow
$ docker logs --tail 100 <id> # last 100 lines
$ docker logs --since 1h <id> # last 1 hour
$ docker logs --timestamps <id> # with time
```



Pro Tips

- ✓ Use -f in production to stream logs.
- ✓ Combine with timestamp for correlation.

3. INSPECT

Inspect returns low-level information about objects.

Common Commands

```
$ docker inspect <container_id> # container
$ docker inspect <image_name> # image
$ docker inspect <volume_name> # volume
$ docker inspect <network_name> # network
```



What to Look For

- ✓ Config, Env, Mounts, Networks
- ✓ IP addresses, Gateway, DNS
- ✓ RestartCount, State, Health
- ✓ Volumes and bind mounts

5. STATS

Real-time resource usage of containers.

Common Commands

```
$ docker stats # all containers
$ docker stats <container_id> # one container
```



Metrics Shown

- | | |
|------------------------|-----------------|
| ✓ CPU % | ✓ Block I/O |
| ✓ Memory Usage / Limit | ✓ PIDs |
| ✓ Net I/O | ✓ Restart Count |

7. VOLUME DEBUGGING

Volumes can cause data, permission, or mount issues.

Common Commands

```
$ docker volume ls
$ docker volume inspect <volume_name>
$ docker exec -it <container> sh
# inside container
ls -l <mount_path>
df -h
```



Common Issues

- | | |
|---------------------|-----------------------|
| ✗ Wrong mount path | ✗ Volume not mounted |
| ✗ Permission denied | ✗ Data not persisting |
| ✗ Disk full | ✗ Driver errors |

2. EXEC

Exec into a running container to run commands.

Common Commands

```
$ docker exec -it <container_id> sh # shell
$ docker exec -it <container_id> bash
$ docker exec <container_id> ps aux # process
$ docker exec <id> env | sort # env vars
```

Use Cases

- ✓ Check running processes
- ✓ Inspect files
- ✓ Test connectivity
- ✓ Run adhoc commands



4. EVENTS

Events show what Docker is doing in real time.

Common Commands

```
$ docker events # all events
$ docker events --since 1h # last 1 hour
$ docker events --filter 'type=container'
$ docker events --filter 'event=start'
```



REAL-TIME
EVENTS

Helpful For

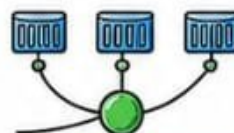
- ✓ Tracking restarts and failures
- ✓ Seeing deploys and removes
- ✓ Correlating issues across services

6. NETWORK DEBUGGING

Check connectivity and network issues.

Common Commands

```
$ docker network ls # list networks
$ docker network inspect <network> # details
$ docker exec -it <container> sh
# then inside container:
ping <ip_or_host>
nslookup <host>
curl http://<host>:<port>
```



CHECK CONNECTIVITY

Things to Check

- | | |
|----------------------------|---------------------|
| ✓ Container IP and gateway | ✓ Port connectivity |
| ✓ DNS resolution | ✓ Service discovery |

8. INCIDENT WORKFLOW

Follow a structured workflow during an incident.



DETECT



OBSERVE



ISOLATE



RESOLVE



VERIFY



DOCUMENT

STEP	ACTION
DETECT	Alert, user report, monitoring, or health check fails.
OBSERVE	Check logs, events, and metrics. Understand the impact.
ISOLATE	Find the failing component. Reproduce if needed.
RESOLVE	Apply the fix (code, config, resource, network, etc.).
VERIFY	Confirm service is healthy and stable.
DOCUMENT	Record root cause, fix, and lessons learned.



Golden Rule

You can't fix what you can't see.
Observe everything.

Observe with data. Debug with clarity.
Fix with confidence.





INCIDENT RESPONSE

☆ Be prepared. Act fast. Minimize impact. Learn and improve. ☆



A structured response limits damage, restores service quickly, and prevents the same issue from happening again.

1. DETECTION

Identify that something is wrong as early as possible.

Detection Sources

- ✓ Monitoring & alerting (CPU, memory, errors)
- ✓ Logs & application metrics
- ✓ User reports or support tickets
- ✓ Security alerts (scans, IDS, SIEM)
- ✓ Health checks & synthetic tests



Key Goals

- ✓ Detect quickly
- ✓ Validate the alert
- ✓ Assess severity and impact

Example Alert

High error rate (5xx) on checkout service for 3+ minutes.

3. INVESTIGATION

Understand what happened, why, and the scope.

Investigation Checklist

- ✓ Review logs, events, and metrics
- ✓ Check config changes and deployments
- ✓ Identify affected services and users
- ✓ Analyse timeline and correlate events
- ✓ Capture evidence (screenshots, logs, configs)



Useful Commands

```
$ docker ps -a          # running & exited containers
$ docker logs <id> --tail 200
$ docker inspect <id>   # config, mounts, env
$ docker events --since 1h
$ docker stats          # resource usage
```



5. POSTMORTEM

Document what happened while it's still fresh.

Postmortem Includes

- ✓ What happened (summary)
- ✓ Impact (who/what was affected)
- ✓ Timeline of events
- ✓ What went well / what didn't
- ✓ Action items and owners



Example: Checkout service outage

Between 10:02-10:18 UTC, users experienced errors during checkout due to a memory leak in v1.5.1. Rolled back to v1.4.2.

7. COMMUNICATION

Keep stakeholders informed with clear and timely updates.

Who to Communicate

- Customers / Users
- Operations / On-call
- Security / Compliance
- Leadership / Management

Communication Channels

- Status page / Incident page
- Slack / Teams / ChatOps
- Email / SMS / Phone
- Support tickets

2. CONTAINMENT

Stop the incident from causing more damage.

Containment Actions

- ✓ Isolate affected container(s) or service(s)
- ✓ Scale down or stop the failing service
- ✓ Block malicious IPs or traffic
- ✓ Revoke credentials / keys if compromised
- ✓ Enable feature flags to disable risky features
- ✓ Apply network rules to limit blast radius



Example

A compromised API container is removed from the load balancer and scaled down to 0. Traffic is routed to healthy instances.

4. RECOVERY

Restore normal service and ensure stability.

Recovery Actions

- ✓ Redeploy or restart healthy instances
- ✓ Apply fix or rollback to last known good
- ✓ Restore data from backup if needed
- ✓ Warm up caches and connections
- ✓ Monitor closely after recovery



Example

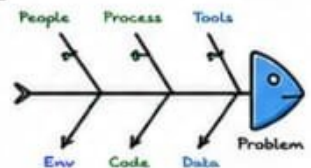
Rollback to previous image: myapp:v1.4.2, restart containers, verify health checks, and monitor.

6. RCA (ROOT CAUSE ANALYSIS)

Find the underlying cause to prevent recurrence.

RCA Techniques

- ✓ 5 Whys
- ✓ Fishbone Diagram (Ishikawa)
- ✓ Timeline Analysis
- ✓ Change Analysis (What changed?)
- ✓ Blameless mindset - focus on systems



5 Whys Example

Problem: API crashed

Why? Memory leak → Why? Unbounded cache → Why? Missing limit
→ Why? Code oversight → Why? No load testing → **Root Cause:**
Insufficient testing for high-load scenarios.

INCIDENT RESPONSE WORKFLOW



Detect



Contain



Investigate



Recover



Postmortem



RCA



Communicate



Incidents will happen. How you respond defines your reliability.
Prepare. Respond. Learn. Improve.





DISASTER RECOVERY

★ Disasters happen. Recovery is a choice. Plan today, survive tomorrow. ★



Disaster Recovery (DR) ensures we can restore critical services quickly after a major failure or outage.

1. BACKUP STRATEGY

Back up everything that matters.

What to Backup

- ✓ Container images
- ✓ Application data (volumes)
- ✓ Databases
- ✓ Configuration files
- ✓ Secrets (external store)
- ✓ Infrastructure as code

Backup Methods

- Registry replication
- Volume snapshots
- Database backups
- Config in Git
- Offsite / Cloud storage



Backup Frequency

- Critical Data → Every 15 mins - 1 hour
- Database → Every 1-4 hours
- Configs/Code → On every change
- Images → On every build



Golden Rule

3-2-1 Rule: 3 copies, 2 different media, 1 offsite.

3. REGISTRY RECOVERY

Ensure your container images are always available.

Strategies

- Use registry replication (active-active or active-passive)
- Regularly export images to object storage
- Keep immutable tags and versioned images
- Use multi-region registries for HA



Example: Backup to S3

```
$ crane cp myregistry/app:1.2.3 s3://dr-backup/app:1.2.3
$ aws s3 sync s3://dr-backup/ s3://dr-backup-replica/
```

5. RECOVERY TESTING

Test today so recovery works tomorrow.

Testing Types

- ✓ Backup restore test (data & configs)
- ✓ Disaster simulation (site/region failure)
- ✓ Failover test (DNS, load balancer)
- ✓ Application validation test
- ✓ Performance test after recovery



Frequency

- Quarterly (minimum)
- ✓ After major changes
- Before critical releases

7. DISASTER RECOVERY CHECKLIST

Use this checklist to stay DR-ready.

- | | |
|--|--|
| ☐ All critical data is identified | ☐ Recovery tests are scheduled |
| ☐ Backup strategy is documented | ☐ RTO and RPO are defined & reviewed |
| ☐ Automated backups are configured | ☐ Communication plan is documented |
| ☐ Backups are stored offsite / cross-region | ☐ Team roles & responsibilities are clear |
| ☐ Registry replication is enabled | ☐ Dependencies and third parties identified |
| ☐ Restore procedures are documented | ☐ Business continuity plan is up to date |
| ☐ Access to backups is tested | ☐ Post-incident review process defined |

2. RESTORE STRATEGY

Practice a clear, repeatable restore process.

Restore Steps

1. Identify what needs to be restored
2. Choose the latest good backup
3. Restore data / configs / images
4. Recreate services and dependencies
5. Verify functionality and integrity
6. Monitor and validate



RESTORE
CONFIDENCE

RTO vs RPO

RTO (Recovery Time Objective): How fast we restore service.

RPO (Recovery Point Objective): How much data we can lose.

Example Targets

- RTO: 1 hour
- RPO: 15 minutes



4. VOLUME RECOVERY

Data lives in volumes. Protect and restore them.

Backup Methods

- ✓ Volume snapshots (LVM, EBS, Ceph, etc.)
- ✓ File system backup (rsync, restic, borg)
- ✓ Database native backup (mysqldump, pg_dump)
- ✓ Offsite replication / cross-region copy



DATA IS
PRECIOUS

Restore Steps

- Attach or mount the backup volume
- Copy data back to new volume
- Start the container / service
- Verify data integrity and application health

Best Practice

- ✓ Test restores regularly.
- A backup is useless if you can't restore it.

6. BUSINESS CONTINUITY

Keep the business running, even in worst-case scenarios.

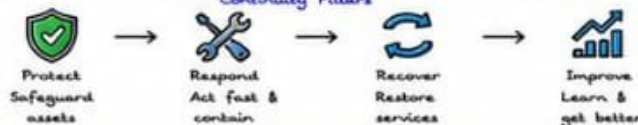
Key Elements

- ✓ Identify critical services and dependencies
- ✓ Define minimum viable service (MVS)
- ✓ Document fallback / manual procedures
- ✓ Maintain communication plan
- ✓ Ensure access to critical tools & credentials



PEOPLE + PLAN +
TECHNOLOGY

Continuity Pillars



DR EXAMPLE FLOW



Example Scenario

Region outage: Primary region us-east-1 fails.

DR Plan: Traffic routed to us-west-2.

Steps: DNS failover → Start services → Restore DB from snapshot → Verify health → Inform stakeholders.



Plan for the worst. Prepare for the best.

Resilience is an engineering advantage.





PRODUCTION OUTAGE CASE STUDIES

☆ Real outages. Real impact. Real lessons. ☆



Outages are inevitable. What matters is how fast we detect, respond, and learn to prevent them again.

1. DISK FULL



Symptoms

- Containers stop writing logs
- App errors: "No space left on device"
- Services become unresponsive

Root Cause

- Unbounded logs and tmp files
- No log rotation
- Large docker images cached

Impact

- New containers cannot start
- DB writes fail
- Full outage for users

What We Did

- Cleaned logs and tmp
- Enabled log rotation
- Set up disk monitoring & alerts

Lesson

Monitor disk usage. Automate cleanup. Alert before 80% usage.

2. MEMORY EXHAUSTION



Symptoms

- OOMKilled containers
- High memory usage
- System becomes slow / unresponsive

Root Cause

- Containers with no memory limits
- Memory leaks in application
- Too many containers on host

Impact

- Containers killed by OOM
- Requests timeout
- Cascading failures

What We Did

- Added memory limits & reservations
- Fixed memory leak in app
- Right-sized instances

Lesson

Always set memory limits. Monitor and profile your apps.

3. CONTAINER CRASH LOOPS



Symptoms

- Container keeps starting and stopping
- Restart count increases
- App never becomes healthy

Root Cause

- Bad configuration
- Missing dependency
- App crashes on start

Impact

- Service unavailable
- High resource churn
- Errors in monitoring

What We Did

- Checked logs with: `docker logs <id>`
- Fixed config / missing env vars
- Added healthcheck & retries

Lesson

Validate configs. Use healthchecks. Fail fast and recover fast.

4. REGISTRY OUTAGE



Symptoms

- Cannot pull images
- Deployments fail
- CI/CD pipeline breaks

Root Cause

- Registry service down
- Network issue to registry
- Storage backend failure

Impact

- New deployments blocked
- Auto-scaling fails
- CI builds fail

What We Did

- Switched to backup registry
- Enabled image caching
- Implemented registry HA

Lesson

Don't rely on a single registry. Cache images and plan failover.

5. NETWORK FAILURE



Symptoms

- Inter-service timeouts
- DNS resolution failures
- "Connection refused / timeout" errors

Root Cause

- Network misconfiguration
- Firewall rule changes
- Upstream ISP outage

Impact

- Services cannot communicate
- External APIs unreachable
- Partial or full outage

What We Did

- Restored network routes
- Fixed DNS / firewall rules
- Added retries & circuit breakers

Lesson

Design for failure. Use retries, timeouts, and multi-AZ networking.

6. SECRET EXPOSURE



Symptoms

- Credentials found in logs / repo
- Alert from secret scanning
- Unauthorized access detected

Root Cause

- Secrets hardcoded in code
- Secrets in image layers
- Overly permissive access

Impact

- Data breach risk
- Account compromise
- Compliance violation

What We Did

- Revoked exposed credentials
- Rotated all related secrets
- Moved secrets to secure store

Lesson

Never hardcode secrets. Use secret managers and rotate.

7. LESSONS LEARNED (ACROSS ALL OUTAGES)

- ✓ Monitor everything (metrics, logs, events)
- ✓ Set alerts with the right thresholds
- ✓ Have runbooks and automate responses
- ✓ Test failure scenarios regularly
- ✓ Document and share knowledge
- ✓ Blameless postmortems drive improvements



OUTAGE RESPONSE FLOW



Alert
Detect
early



Triage
Understand
impact



Contain
Stop the
bleeding



Fix
Resolve
issue



Review
Postmortem
& RCA



Improve
Prevent
recurrence

OUTAGE IMPACT EXAMPLES

Severity	Users Impacted	Service Availability	Response Target
SEV 1 (Critical)	All users	0%	< 15 minutes
SEV 2 (High)	Majority	1% - 50%	< 1 hour
SEV 3 (Medium)	Some users	50% - 90%	< 4 hours
SEV 4 (Low)	Few users	90% - 99%	< 24 hours

EXAMPLE: E-COMMERCE OUTAGE TIMELINE

10:02 Alert: High 5xx error rate
10:03 Triage: Orders failing, checkout down
10:05 Contain: Disabled new deployments
10:12 Fix: Restored DB space, cleaned logs
10:25 Monitor: Service healthy
10:40 Postmortem started



Outages will happen. Resilience is built in advance.

Plan. Monitor. Respond. Learn. Improve.



ADVANCED PRODUCTION INTERVIEW QUESTIONS

★ Real-world problems. Real scenarios. Production mindset. ★



These questions test your production experience, troubleshooting skills, and ability to design reliable Docker solutions.

1. 50 PRODUCTION-FOCUSED DOCKER INTERVIEW QUESTIONS

- What happens when a container starts?
- How does Docker networking work?
- Explain bridge vs host vs none network.
- How do you persist data in Docker?
- What is the difference between COPY and ADD in Dockerfile?
- How do you run a container as non-root?
- How do you limit container resources?
- What is a Docker health check?
- How do you handle secrets in Docker?
- How do you reduce Docker image size?
- How do you scan images for vulnerabilities?
- What is a multi-stage build?
- How do you handle zero-downtime deployments with Docker?
- How do you debug a failing container?
- How do you collect logs from containers?
- How do you monitor container performance?
- How do you troubleshoot a container that exits immediately?
- How do you handle container restart policies?
- What is the difference between CMD and ENTRYPOINT?
- How do you backup Docker volumes?
- How do you restore Docker volumes?
- How do you recover from a corrupted volume?
- How do you troubleshoot disk space issues?
- How do you secure the Docker daemon?
- What is the default docker0 network?
- How do you connect containers across hosts?
- How do you troubleshoot DNS issues in containers?
- How do you update a running container?
- How do you rollback a failed deployment?
- How do you handle blue/green deployments?
- How do you secure a Docker registry?
- How do you mirror a Docker registry?
- How do you handle private base images?
- How do you prevent container breakout attacks?
- How do you audit Docker events?
- How do you troubleshoot slow containers?
- How do you debug networking latency?
- How do you handle high CPU containers?
- How do you debug memory leaks?
- How do you handle containers in read-only FS?
- How do you troubleshoot permission errors?
- How do you debug volume mount issues?
- How do you troubleshoot port conflicts?
- How do you handle configuration management?
- How do you implement log rotation?
- How do you secure container images?
- How do you handle compliance requirements?
- How do you design highly available apps with Docker?
- How do you handle disaster recovery?
- How do you document your Docker environment?

2. SCENARIO-BASED QUESTIONS

Scenario 1: App is Down

Q: Your web app container is not responding. What do you do?

A: Check container status → logs → resource usage → health check → network → dependencies → restart/rollback.



Scenario 2: Disk Full

Q: Docker host disk is full. What steps do you take?

A: Check disk usage → docker system df → prune unused images/containers/volumes → rotate logs → alert & monitor.



Scenario 3: High CPU

Q: A container is using 100% CPU. What will you do?

A: Check top/htop → docker stats → inspect container → check app logs → find runaway process → limit resources.



Scenario 4: Deployment Failed

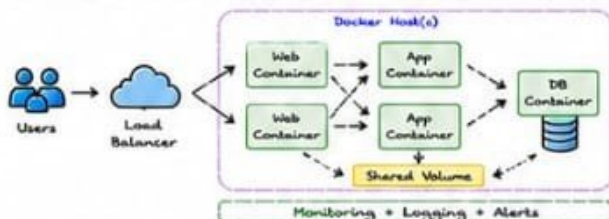
Q: New version deployment failed and users see errors.

A: Check logs → health checks → rollback to previous version → fix issue → test → redeploy.



4. ARCHITECTURE QUESTIONS

Q1: Design a scalable, highly available Docker-based web application.



Q2: How will you ensure zero downtime during deployments?

A: Use rolling update strategy → health checks → load balancer → blue/green or canary → rollback on failure.



INTERVIEW TIPS

- Think out loud.
- Explain your thought process.
- Share real production examples.
- Show how you prevent issues, not just fix them.



3. TROUBLESHOOTING QUESTIONS

PROBLEM	WHAT TO CHECK	TOOLS/COMMANDS
Container not starting	Logs, config, ports, health, resources	docker logs, docker inspect, docker ps -a
App not reachable	Network, DNS, firewall, port mapping	docker network, ls, nslookup, curl, telnet
High memory usage	Memory leaks, limits, unexpected processes	docker stats, free -h, ps aux
Slow performance	CPU, IO, network, app bottlenecks	docker stats, iostat, top, dstat
Volume not mounted	Path, permissions, volume driver	docker inspect, ls -l, df -h
Permission denied	User/Group, file perms, privileged access	ls -l, id, chown, user mapping
Port already in use	Conflicting service, container mapping	netstat -tulnp, lsof -i :PORT

5. MODEL ANSWERS (SAMPLE)

Q: How do you reduce Docker image size?

A: Use multi-stage builds, slim base images, remove unnecessary packages, clean apt caches, use .dockerignore, and avoid copying unneeded files.



Q: How do you secure Docker in production?

A: Run containers as non-root, use read-only FS, limit capabilities, scan images, use secrets, update regularly, secure daemon, and monitor logs.



Q: How do you recover from a container failure?

A: Check logs → identify root cause → fix → restart or replace container → verify health → monitor → update runbooks.



KEY SKILLS TESTED

- Troubleshooting
- System Design
- Performance Tuning
- Security & Reliability



Remember

It's not about knowing every command.
It's about solving **real problems** in production.



Be prepared. Stay calm. Think production.
You've got this!





PRODUCTION ENGINEER CHEAT SHEET



Essential commands. Fast lookup. Production ready.



Bookmark this page. These commands will save you hours in production.

Know them. Use them. Automate them.



1. BUILDKIT COMMANDS



PURPOSE	COMMAND	EXAMPLE
Build with BuildKit	DOCKER_BUILDKIT=1 docker build .	DOCKER_BUILDKIT=1 docker build -t app:1.0 .
Build with progress	docker buildx build --progress=plain .	docker buildx build --progress=plain .
Multi-platform build	docker buildx build --platform linux/amd64, linux/arm64 -t app:1.0 .	docker buildx build --platform linux/amd64, linux/arm64 -t app:1.0 .
Build and push	docker buildx build --push -t repo/app:1.0 .	docker buildx build --push -t repo/app:1.0 .
BuildKit prune	docker builder prune -f	docker builder prune -f
List builders	docker buildx ls	docker buildx ls
Inspect builder	docker buildx inspect --bootstrap	docker buildx inspect --bootstrap

3. HEALTH CHECK COMMANDS



PURPOSE	COMMAND	EXAMPLE
Show health status	docker inspect --format '{{.State.Health.Status}}' <id>	docker inspect --format '{{.State.Health.Status}}' web_1
Show health logs	docker inspect --format '{{json .State.Health.Log}}' <id>	docker inspect --format '{{json .State.Health.Log}}' web_1
View container health in list	docker ps --format 'table {{.Names}} t{{.Status}}'	docker ps --format 'table {{.Names}} t{{.Status}}'
Test curl health	curl -f http://localhost:8080/ health echo 'FAIL'	curl -f http://localhost:8080/ health echo 'FAIL'
Test TCP port	nc -zv host port	nc -zv localhost 3306

5. LOGGING COMMANDS



PURPOSE	COMMAND	EXAMPLE
View logs	docker logs <id>	docker logs web_1
Follow logs	docker logs -f <id>	docker logs -f web_1
Tail last N lines	docker logs --tail 100 <id>	docker logs --tail 100 web_1
Since time	docker logs --since 1h <id>	docker logs --since 1h web_1
Since timestamp	docker logs --since 2024-01-01T10:00:00 <id>	docker logs --since 2024-01-01T10:00:00 web_1
Logs with timestamps	docker logs -t <id>	docker logs -t web_1
Logs to file	docker logs <id> > app.log	docker logs web_1 > app.log

7. RECOVERY COMMANDS



PURPOSE	COMMAND	EXAMPLE
Restart container	docker restart <id>	docker restart web_1
Stop container	docker stop <id>	docker stop web_1
Remove container	docker rm <id>	docker rm web_1
Prune stopped containers	docker container prune -f	docker container prune -f
Prune system	docker system prune -a -f	docker system prune -a -f
Recreate from image	docker run -d --name web_1 -p 80:80 app:1.0	docker run -d --name web_1 -p 80:80 app:1.0
Rollback to previous image	docker run -d --name web_1 app:prev-version	docker run -d --name web_1 app:v1.2.3

★ PRO TIP

- ✓ Automate everything you can.
- ✓ Monitor everything that matters.
- ✓ Test failures before they happen.

2. TRIVY COMMANDS



PURPOSE	COMMAND	EXAMPLE
Scan image	trivy image <image>	trivy image nginx:1.25
Scan image (severity)	trivy image --severity HIGH,CRITICAL <image>	trivy image --severity HIGH,CRITICAL nginx:1.25
Scan filesystem	trivy fs <path>	trivy fs .
Scan with SBOM	trivy image --format cyclonedx -o sbom.xml 	trivy image --format cyclonedx -o sbom.xml app:1.0
Scan config (IaC)	trivy config <file>	trivy config docker-compose.yml
Ignore unpatched	trivy image --ignore- unfixed <image>	trivy image --ignore- unfixed app:1.0
Output as table	trivy image --format table <image>	trivy image --format table nginx:1.25
Update DB	trivy --download-db-only	trivy --download-db-only

4. MONITORING COMMANDS



PURPOSE	COMMAND	EXAMPLE
Container stats (live)	docker stats	docker stats
Stats for specific container	docker stats <id>	docker stats web_1
One-shot stats	docker stats --no-stream	docker stats --no-stream
System info	docker system df	docker system df
Events (real-time)	docker events	docker events
Inspect resource usage	docker inspect <id> --format='{{.State}}'	docker inspect web_1 --format='{{.State}}'
Top processes in container	docker top <id>	docker top web_1

6. SECURITY COMMANDS



PURPOSE	COMMAND	EXAMPLE
Check capabilities	docker inspect --format '{{.HostConfig.CapAdd}}' <id>	docker inspect --format '{{.HostConfig.CapAdd}}' web_1
Read-only root fs?	docker inspect --format '{{.HostConfig.ReadOnlyRootfs}}' <id>	docker inspect --format '{{.HostConfig.ReadOnlyRootfs}}' web_1
User in container	docker inspect --format '{{.Config.User}}' <id>	docker inspect --format '{{.Config.User}}' web_1
Check exposed ports	docker inspect -f '{{.Config.ExposedPorts}}' <id>	docker inspect -f '{{.Config.ExposedPorts}}' web_1
Image scan (trivy)	trivy image <image>	trivy image app:1.0
Check secrets in env	docker inspect --format '{{.Config.Env}}' <id>	docker inspect --format '{{.Config.Env}}' web_1

8. QUICK REFERENCE TABLES



Common Exit Codes

0	Success
1	General error
125	Docker CLI error
126	Permission denied
127	Command not found
137	Killed (OOM)
143	Terminated (SIGTERM)

Log Drivers

json-file	(default)
local	(better for prod)
syslog	(send to syslog)
journald	(systemd)
fluentd	(forward to fluentd)
awslogs	(CloudWatch Logs)
gelf	(Graylog)

Restart Policies

no	: Never restart
on-failure	: Restart if container exits with error
always	: Always restart
unless-stopped	: Always restart unless manually stopped

Health Status Values

● healthy
● unhealthy
● starting
● none

Great engineers don't just fix problems,
they prevent them.



PRODUCTION READINESS CHECKLIST

Use this checklist before every production deployment.

If you can't check it, don't ship it.



A checklist is your safety net.

Great systems are prepared. Not just deployed.



1. DEPLOYMENT



- ☒ Image built with BuildKit
- ☒ Image tagged with version (no :latest)
- ☒ Image scanned (Trivy) - no critical issues
- ☒ Deployment manifests version controlled
- ☒ Environment variables externalized
- ☒ Secrets managed (no hardcoded values)
- ☒ Rolling update strategy configured
- ☒ Rollback strategy tested
- ☒ Deployment pipeline automated
- ☒ Smoke test after deployment
- ☒ Feature flags used for risky changes

2. SECURITY



- ☒ Base image is minimal (Alpine/Distroless)
- ☒ Image scanned (Trivy) - no high/critical
- ☒ Non-root user configured
- ☒ Container dropped unnecessary capabilities
- ☒ Read-only root filesystem (where possible)
- ☒ Secrets stored in Docker Swarm/K8s secrets or external vault
- ☒ Network access restricted
- ☒ Unnecessary ports closed
- ☒ Security policies (AppArmor/SELinux) set
- ☒ Regular dependency updates enabled
- ☒ Audit logs enabled

3. MONITORING



- ☒ Container metrics (CPU, Memory) collected
- ☒ Service availability monitored
- ☒ Custom application metrics exposed
- ☒ Alerts configured (thresholds set)
- ☒ Dashboard created (Grafana/Prometheus)
- ☒ Uptime checks configured
- ☒ SLOs / SLIs defined and tracked
- ☒ Resource saturation alerts enabled
- ☒ Health endpoints monitored
- ☒ Alert routing to on-call (PagerDuty/Slack)
- ☒ Runbooks linked in alerts

4. LOGGING



- ☒ Application logs to STDOUT/STDERR
- ☒ Centralized logging configured
- ☒ Structured logging (JSON) enabled
- ☒ Log levels configured (info/warn/error)
- ☒ Sensitive data not logged
- ☒ Log retention policy configured
- ☒ Log search & alerting enabled
- ☒ Audit trail logs enabled
- ☒ Timestamps in UTC
- ☒ Correlation/Request IDs in logs

5. BACKUP



- ☒ Databases have automated backups
- ☒ Volumes/State data backed up
- ☒ Backup stored off-host / off-region
- ☒ Backup encryption enabled
- ☒ Backup integrity verification scheduled
- ☒ Backup retention policy defined
- ☒ Backup restore procedure documented
- ☒ Backup monitoring & alerts enabled
- ☒ Configuration files backed up
- ☒ Secrets & certificates backed up

6. DISASTER RECOVERY



- ☒ Multi-AZ / multi-region strategy defined
- ☒ RPO (Recovery Point Objective) defined
- ☒ RTO (Recovery Time Objective) defined
- ☒ Disaster recovery runbook created
- ☒ Data replication configured & tested
- ☒ Failover procedure tested
- ☒ Communication plan defined
- ☒ DR contacts & escalation path defined
- ☒ Regular DR drills performed
- ☒ Critical dependencies identified

7. PERFORMANCE



- ☒ Load tested with realistic traffic
- ☒ Response time meets requirements
- ☒ Database queries optimized
- ☒ Caching strategy implemented
- ☒ Static assets served via CDN (if applicable)
- ☒ Connection pooling configured
- ☒ Resource contention tested
- ☒ Performance baseline established
- ☒ Bottlenecks identified & addressed
- ☒ Autoscaling rules defined (if applicable)

8. RESOURCE LIMITS



- ☒ CPU limit set
- ☒ Memory limit set
- ☒ Requests & limits tuned
- ☒ PIDs limit configured
- ☒ Storage/Volume size monitored
- ☒ OOMKilled handling strategy tested
- ☒ Ulimit settings reviewed
- ☒ Node capacity monitored
- ☒ Resource quotas set (cluster level)
- ☒ Resource alerts configured

9. HEALTH CHECKS



- ☒ Docker HEALTHCHECK configured
- ☒ Health endpoint is lightweight
- ☒ Startup, interval, timeout, retries set
- ☒ Health check dependency safe
- ☒ Unhealthy threshold tested
- ☒ Service removed from load balancer if unhealthy
- ☒ Health check logs monitored
- ☒ Health check failures alerting enabled
- ☒ Liveness & Readiness validated
- ☒ Flapping health issues resolved

10. FINAL PRODUCTION SIGN-OFF



SIGN-OFF MATRIX

ROLE	NAME	SIGNATURE	DATE
Dev / Engineer			
QA / Tester			
Security			
Ops / SRE			
Product Owner			

DEPLOYMENT GATE

Ask these final questions:

- Is the system reliable?
- Is it secure?
- Can we monitor it?
- Can we recover if it fails?
- Have we tested failure scenarios?

If you answered YES to all,
You are ready to deploy!



PRODUCTION CHECK FLOW



Plan & Build
↓
Secure & Configure
↓
Monitor & Log
↓
Backup & Prepare
↓
Recover & Test
↓
Deploy with Confidence



Prepared today. Protected tomorrow.

Production excellence is a habit, not an accident.

